

Network Traffic Forensics

Packets · protocols · streams · covert channels

Stefanus Eko Prasetyo

Lab-5 · hands-on session

```
root@dfir:~/lab05#  
  
$ tshark -r challenge.pcapng  
-Y 'tcp.port==5555'  
11 packets - something odd here  
# the wire remembers everything
```

Today's roadmap

1

Networking refresher

IP addresses, protocols, and ports - the coordinates of every conversation

2

Capturing traffic

Wireshark and tcpdump; pcap vs pcapng

3

Analysis core skills

protocol hierarchy, display filters, packet details and bytes

4

Streams and exports

Follow TCP Stream and Export Objects - where evidence surfaces

5

Attack patterns on the wire

brute force, plaintext credentials, reverse shells, covert channels

6

Challenges & working paper

upstream breach capture + two new challenges + documentation rules

The network never forgets

Disks can be wiped and logs cleared, but a captured packet is a **verbatim recording of a conversation** - headers, payloads, timing, everything. This lab continues the Lab 04 breach: the same attacker, seen this time from the wire.

Who and where

source and destination IPs identify the talkers

What

protocol and ports reveal the service - and the oddities

Said what

payloads carry commands, credentials, and files

When

microsecond timestamps build the sequence of events

IP, protocol, port - the coordinates

Protocol	Port	Protocol	Port
FTP	21	HTTP	80
SSH	22	HTTPS	443
Telnet	23	SMB	445
SMTP	25	RDP	3389
DNS	53	MySQL	3306
DHCP	67/68	ICMP	n/a (no ports)

A conversation = (source IP : port) to (destination IP : port) over a protocol. Memorize the common ports: unexpected ports (4444, 5555) are instant suspicion, and known ports carry the services you can read in plaintext - FTP, HTTP, Telnet.

Recording the wire

```
root@dfir:~/lab05#  
  
# live capture with Wireshark:  
$ wireshark          # pick interface, click the fin  
  
# headless with tcpdump:  
$ sudo tcpdump -i eth0 -w capture.pcap  
  
# capture file formats:  
.pcap   classic libpcap  
.pcapng pcap next generation (default in Wireshark)  
# Wireshark opens both transparently
```

Capture discipline

- ▶ record interface + times; captures carry timezones
- ▶ save immediately; work on COPIES of evidence captures
- ▶ hash the capture before and after analysis
- ▶ full-packet captures include payloads - treat them as confidential

Three panes, one question at a time

Pane	Shows	You use it to
Packet list	one line per packet: number, time, src, dst, protocol, info	scan, sort, and spot anomalies
Packet details	the protocol layers of the selected packet, decoded	drill into headers field by field
Packet bytes	the raw hexdump with ASCII rendering	see EXACTLY what traveled, byte by byte

Pro habit: right-click any field in the details pane - Apply as Filter and Follow -> Stream turn a field into a query instantly.

Protocol hierarchy - orient first

```
root@dfir:~/lab05#  
  
Statistics -> Protocol Hierarchy  
  
Frame      2938  100%  
Ethernet    2938  
IPv4        2938  
  TCP       2898   98.6%  
    HTTP     78    TLS 45   MySQL 42  
  ICMP           40  
  
# questions to ask: expected mix? unknown ports? TLS to  
# odd hosts? 2500 FTP packets = one very busy session
```

Why hierarchy first

- ▶ one screen summarizes the whole capture
- ▶ percentages point you to the bulk
- ▶ unexpected protocols surface immediately
- ▶ then filter - never scroll 3,000 packets blind

Display filters - the query language

Goal	Filter
One protocol	http ftp icmp dns
One service/port	tcp.port == 21 tcp.port == 5555
One talker	ip.addr == 192.168.0.106
Login attempts	ftp.request.command == "PASS"
Combine	http && ip.addr == 10.1.1.77
Large transfers	tcp.len > 500

Autocomplete as you type; the status bar shows how many packets survived. Filter != deletion - everything is still there.

Details and bytes - reading one packet

```
root@dfir:~/lab05#  
  
> Frame 2674: 386 bytes on wire  
> Ethernet II, Src 02:00:00:00:01:77  
> Internet Protocol Version 4, Src: 192.168.0.106  
> Transmission Control Protocol, Src Port: 48106  
> Hypertext Transfer Protocol  
    POST /command.php HTTP/1.1  
    ... cmd=bash+-i+%3E%26 ... (URL-encoded!)  
  
# the bytes pane shows the same data as hex+ASCII;  
# what you see there is what crossed the wire
```

Reading layers

- ▶ expand one layer at a time - top to bottom
- ▶ each layer answers one question (who? how? what?)
- ▶ bytes pane = ground truth for payloads
- ▶ copy values with right-click - no retyping hex

Follow TCP Stream - the whole conversation

```
root@dfir:~/lab05#  
  
# right-click a packet -> Follow -> TCP Stream  
  
220 (vsFTPd 3.0.5)  
USER ftp  
331 Please specify the password.  
PASS batman  
230 Login successful.  
  
# one direction red, one blue - who said what.  
# stream numbers organize the capture into chats.
```

Why streams win

- ▶ reassembles both directions in order
- ▶ plaintext protocols read like a chat log
- ▶ the stream number becomes your citation
- ▶ works for HTTP, FTP, Telnet - and odd ports

Export Objects - files that crossed the wire

```
root@dfir:~/lab05#  
  
File -> Export Objects -> HTTP  
  
Filename      Content-Type  Size  Packets  
index.html    text/html    720   12-18  
roster.txt   text/plain  487  40-46  
  
# Save As... writes the recovered file to disk  
# hash it - it is evidence now
```

Notes

- ▶ also works for SMB, FTP-DATA, DICOM...
- ▶ reassembles chunked transfers for you
- ▶ combine with streams: which page linked the file?
- ▶ exported malware = handle carefully

What attacks look like in a capture

Pattern	Wire signature
Brute force	many short streams to one port; repeated USER/PASS, 530 responses
Credential leak	plaintext logins (FTP/HTTP/Telnet) readable in Follow Stream
Web exploitation	POST payloads with encoded commands; odd parameters
Reverse shell	odd high port (4444); one side types commands, other echoes output
Data theft	large outbound transfers; file exports; unexpected TLS to file hosts
Covert channel	pings with non-standard payloads; hex/base64 where ASCII should be

Covert channels - hiding inside the normal

A covert channel smuggles data through traffic that looks legitimate: pings, DNS queries, even timing.
The protocol is normal; the payload is not.

```
root@dfir:~/lab05#  
  
# ICMP echo should carry abcdefg... padding:  
Data (32 bytes): ab cd ef 12 34 ...  
# a covert one carries something else:  
Data (33 bytes): 23 59 47 56 73 6a ... ('#YGVsj...')  
# reassemble request payloads in order, then decode
```

Hunting rule

- ▶ know what NORMAL looks like first
- ▶ compare payload patterns across packets
- ▶ requests vs replies: who initiates?
- ▶ sequence numbers define the order

The traffic-analysis toolbox

Tool	Purpose
Wireshark	interactive analysis: hierarchy, filters, streams, exports
tshark	the same power scripted: -Y filters, -z statistics, -T fields
tcpdump	lightweight capture and quick inspection
capinfos	capture metadata: times, counts, hashes - your intake report
Follow TCP Stream	reassemble conversations - the evidence transcript
Export Objects	recover transferred files without manual reassembly

Your challenges

Challenge A

the breach capture

`files/challenge.pcapng`

- ▶ 2,938 packets, one attacker
- ▶ 10 questions, 2 flags
- ▶ FTP, shell, root, backdoor
- ▶ solved together in the Deep Dive

from the original lab

Challenge B — new

`office_hours.pcap`

`files/office_hours.pcap`

- ▶ a quiet intranet capture
- ▶ a login that eventually works
- ▶ a file worth exporting
- ▶ the token inside is encoded

streams & exports

Challenge C — new

`ping_puzzle.pcap`

`files/ping_puzzle.pcap`

- ▶ 112 packets of pure ICMP
- ▶ one host keeps pinging
- ▶ payloads are not padding
- ▶ reassemble, then decode

covert channel hunt

Flag format: `ctf{xxxx}` (Challenge A keeps the upstream flag format)

Working paper & evidence rules

1 Pre-lab first

section 1.4 of the working paper, before the session

2 Work on copies

copy the pcap; hash the original before and after

3 Cite packet numbers

answers name the packet number, stream number, or filter used

4 Screenshot the stream

evidence = the followed stream, not a paraphrase

5 Treat payloads as live

malware URLs and keys in captures are secrets - do not leak them

```
root@dfir:~/lab05#  
# evidence log entry  
[B-04] 2026-10-19 09:12 Wireshark  
Follow TCP Stream #3  
    PASS batman -> 230 Login successful  
-> evidence/B-04_ftp_stream3.png
```

No stream, no answer.

every claim cites a packet or stream number

KEY TAKEAWAYS

01

Orient before you scroll

protocol hierarchy and conversations summarize thousands of packets

02

Streams are the transcript

Follow TCP Stream turns a capture into readable conversations

03

Plaintext protocols confess

FTP, HTTP, Telnet carry credentials in the clear - always check

04

Odd ports hide stories

4444 and 5555: reverse shells and covert channels live there

Listen to the wire.

root@dfir:~/lab05#

```
$ git clone https://github.com/stprasst/digital-forensic-lab
$ cd digital-forensic-lab/Lab\ 05/files
$ ls *.pcap*
capture.pcapng  challenge.pcapng  office_hours.pcap  ping_puzzle.pcap
```